

Extended ACLs

- Topics covered:
 - Another Way to Configure Numbered ACLs
 - Editing ACLs
 - Extended Numbered and Named ACLs

Another Way to Configure Numbered ACLs

Configuring Numbered ACLs (Traditional Method)

Previously, numbered ACLs were configured directly from global configuration mode using the access-list command.

CLI Output:

```
R1(config)# access-list 1 deny 192.168.1.1
R1(config)# access-list 1 permit any
```

Explanation:

- ACL 1 denies traffic from 192.168.1.1.
- All other traffic is permitted.
- Entries are configured directly in global config mode.

Configuring Named ACLs

Named ACLs use a separate ACL configuration mode.

CLI Output:

```
R1(config)# ip access-list standard BLOCK_PC1
R1(config-std-nacl)# deny 192.168.1.1
R1(config-std-nacl)# permit any
```

Explanation:

- ip access-list standard BLOCK_PC1 enters named ACL config mode.
- Entries are added as subcommands inside that mode.

Modern IOS – Numbered ACLs Using Named ACL Method

In modern IOS, numbered ACLs can also be configured using the same method as named ACLs.

CLI Output:

```
R1(config)# ip access-list standard 1
R1(config-std-nacl)# deny 192.168.1.1
R1(config-std-nacl)# permit any
```

Important Note:

Even though it is configured in named ACL mode, when checking the running configuration it appears as a traditional numbered ACL.

CLI Output:

```
R1(config)#ip access-list standard ?
<1-99>      Standard IP access-list number
<1300-1999> Standard IP access-list number (expanded range)
WORD        Access-list name

R1(config)#ip access-list standard 1
R1(config-std-nacl)#deny 192.168.1.1
R1(config-std-nacl)#permit any
R1(config-std-nacl)#
R1(config-std-nacl)#do show running-config | section access-list
access-list 1 deny 192.168.1.1
access-list 1 permit any
R1(config-std-nacl)#
```

Why Use Named ACL Config Mode for Numbered ACLs?

It provides editing advantages.

Advantage 1 – Delete Individual Entries

ACL Before Deletion:

```
R1(config-std-nacl)#do show access-lists
Standard IP access list 1
 10 deny 192.168.1.1
 20 deny 192.168.1.2
 30 deny 192.168.3.0, wildcard bits 0.0.0.255
 40 permit any
```

- Default sequence numbers: 10, 20, 30, 40
- Increase by 10 automatically.

Delete Entry 30:

```
R1(config-std-nacl)#no 30
```

After Deletion:

```
R1(config-std-nacl)#do show access-lists
Standard IP access list 1
 10 deny 192.168.1.1
 20 deny 192.168.1.2
 40 permit any
R1(config-std-nacl)#
```

- Only entry 30 is removed, this makes ACL editing easy.

Traditional Numbered ACL Editing Problem:

Using global config mode:

```
R1(config)#do show access-lists
Standard IP access list 1
 10 deny 192.168.1.1
 20 deny 192.168.1.2
 30 deny 192.168.3.0, wildcard bits 0.0.0.255
 40 permit any
R1(config)#do show running-config | section access-list
access-list 1 deny 192.168.1.1
access-list 1 deny 192.168.1.2
access-list 1 deny 192.168.3.0 0.0.0.255
access-list 1 permit any
```

Attempt to delete one entry:

```
R1(config)#no access-list 1 deny 192.168.3.0 0.0.0.255
```

After checking:

```
R1(config)#do show access-lists
R1(config)#do show running-config | section access-list
R1(config)#
```

Nothing appears.

Result:

The entire ACL was deleted.

When using global config mode, you cannot delete individual entries.

You must delete and recreate the whole ACL.

Advantage 2 – Insert Entries in Between

In named ACL mode, you can manually specify sequence numbers.

ACL Before Insertion:

```
R1(config-std-nacl)#do show access-lists
Standard IP access list 1
 10 deny 192.168.1.1
 20 deny 192.168.1.2
 40 permit any
```

Insert New Entry with Sequence 30:

```
R1(config-std-nacl)#30 deny 192.168.2.0 0.0.0.255
```

After Insertion:

```
R1(config-std-nacl)#do show access-lists
Standard IP access list 1
 10 deny 192.168.1.1
 20 deny 192.168.1.2
 30 deny 192.168.2.0, wildcard bits 0.0.0.255
 40 permit any
R1(config-std-nacl)#
```

Entry inserted between 20 and 40.

In global config mode:

- You cannot choose sequence number.
- New entry is always added at the end.

Editing ACLs

ACL Resequencing

Resequencing fixes poor sequence numbering.

Problem Example:

```
R1(config)#do show access-lists
Standard IP access list 1
  1 deny 192.168.1.1
  3 deny 192.168.3.1
  2 deny 192.168.2.1
  4 deny 192.168.4.1
  5 permit any
```

Sequence numbers: 1,2,3,4,5
No space to insert new entries.

Resequencing Command:

`ip access-list resequence acl-id starting-seq-num increment`

```
R1(config)#ip access-list resequence 1 10 10
```

Command Meaning:

- 1 → ACL number
- First 10 → Starting sequence number
- Second 10 → Increment value

After Resequencing:

```
R1(config)#do show access-lists
Standard IP access list 1
  10 deny 192.168.1.1
  20 deny 192.168.3.1
  30 deny 192.168.2.1
  40 deny 192.168.4.1
  50 permit any
```

Extended Numbered and Named ACLs

Extended ACLs

Extended ACLs work like standard ACLs, but they are more powerful. They are processed from top to bottom, just like standard ACLs. The key difference is that extended ACLs can match traffic using:

1. Protocol (TCP, UDP, ICMP, etc.)
2. Source IP address
3. Destination IP address
4. Port numbers (Layer 4)

Because of this, extended ACLs allow very specific traffic control.

Extended ACL Number Ranges:

Extended numbered ACLs use these ranges:

- 100–199
- 2000–2699

You must remember these.

(Standard ACLs use 1–99 and 1300–1999.)

Basic Extended ACL Command

From Global Configuration Mode:

```
R1(config)# access-list number [permit | deny] protocol src-ip dest-ip
```

Here,

- *number* must be in 100–199 or 2000–2699.
- Then you specify permit or deny.
- Then the protocol (like tcp or udp).
- Then source and destination IP addresses.

Using Named Method (Recommended):

```
R1(config)# ip access-list extended {name | number}
R1(config-ext-nacl)# [seq-num] [permit | deny] protocol src-ip dest-ip
```

This enters extended ACL configuration mode.

Inside this mode, you add individual ACL entries.

Numbered extended ACLs can also be configured using this named method.

Matching Based on Protocol

```
R1(config)#ip access-list extended EXAMPLE
R1(config-ext-nacl)#deny ?
<0-255>      An IP protocol number
ahp          Authentication Header Protocol
eigrp        Cisco's EIGRP routing protocol
esp          Encapsulation Security Payload
gre          Cisco's GRE tunneling
icmp         Internet Control Message Protocol
igmp         Internet Gateway Message Protocol
ip           Any Internet Protocol
ipinip       IP in IP tunneling
nos          KA9Q NOS compatible IP over IP tunneling
object-group Service object group
ospf         OSPF routing protocol
pcp          Payload Compression Protocol
pim          Protocol Independent Multicast
sctp         Stream Control Transmission Protocol
tcp          Transmission Control Protocol
udp          User Datagram Protocol
```

Explanation:

When you type deny ?, IOS shows protocol options.

You can match protocol in two ways:

1. By number (IPv4 protocol field)

- 1 → ICMP
- 6 → TCP
- 17 → UDP
- 88 → EIGRP
- 89 → OSPF

2. By name (more common)

- tcp
- udp
- icmp
- ip

If you use ip, it matches all IP packets, regardless of TCP or UDP.
That is why permit ip any any means allow all traffic.

Adding Source and Destination

Step 1 – Choose Protocol

Example:

```
R1(config-ext-nacl)#deny tcp ?
A.B.C.D      Source address
any          Any source host
host         A single source host
object-group Source network object group
```

After selecting tcp, IOS asks for the source.

You can use:

- any → any source
- host 1.1.1.1 → one specific host
- network + wildcard mask

Step 2 – Destination Options

```
R1(config-ext-nacl)#deny tcp any ?
A.B.C.D      Destination address
any          Any destination host
eq           Match only packets on a given port number
gt           Match only packets with a greater port number
host         A single destination host
lt           Match only packets with a lower port number
neq          Match only packets not on a given port number
object-group Destination network object group
range        Match only packets in the range of port numbers
```

After specifying the source, you define the destination.

Important rule in extended ACLs:

If you want to specify a single host (/32), you must:

- Use host, OR
- Use wildcard mask 0.0.0.0

You cannot just write the IP alone.

Complete Example

```
R1(config-ext-nacl)#deny tcp any 10.0.0.0 ?  
A.B.C.D Destination wildcard bits  
  
R1(config-ext-nacl)#deny tcp any 10.0.0.0 0.0.0.255  
R1(config-ext-nacl)#
```

This entry:

- Matches TCP packets
- From any source
- Going to network 10.0.0.0/24
- And denies them

This is only one ACL entry. A full ACL usually has multiple entries.

Practice Examples with Explanation

1. Allow all traffic

```
R1(config-ext-nacl)#permit ip any any
```

Explanation:

- ip → match all IP protocols
- any any → any source and any destination

This is similar to permit any in a standard ACL.

2. Prevent 10.0.0.0/16 from sending UDP traffic to 192.168.1.1/32

```
R1(config-ext-nacl)#deny udp 10.0.0.0 0.0.255.255 host 192.168.1.1
```

Explanation:

- Protocol = UDP
- Source = 10.0.0.0/16
- Destination = single host 192.168.1.1
- Result → UDP traffic from that network to that host is denied.

3. Prevent 172.16.1.1/32 from pinging hosts in 192.168.0.0/24

```
R1(config-ext-nacl)#deny icmp host 172.16.1.1 192.168.0.0 0.0.0.255
```

Explanation:

- Protocol = ICMP
- Source = 172.16.1.1
- Destination = 192.168.0.0/24
- Effect → That host cannot ping that network.

Matching TCP/UDP Port Numbers

If you specify TCP or UDP, you can also match port numbers.

If you do not specify a port:

- All ports are matched.

Port Matching Options:

```
R1(config-ext-nacl)#deny tcp src-ip eq src-port-num dest-ip eq dst-port-num
                                gt
                                lt
                                neq
                                range
                                gt
                                lt
                                neq
                                range
```

- **eq 80** = equal to port 80
- **gt 80** = greater than 80 (81 and greater)
- **lt 80** = less than 80 (79 and less)
- **neq 80** = NOT 80
- **range 80 100** = from port 80 to port 100

Most used: eq

Common Port Numbers:

TCP	UDP
• FTP data (20)	• DHCP server (67)
• FTP control (21)	• DHCP client (68)
• SSH (22)	• TFTP (69)
• Telnet (23)	• SNMP agent (161)
• SMTP (25)	• SNMP manager (162)
• HTTP (80)	• Syslog (514)
• POP3 (110)	
• HTTPS (443)	
	TCP & UDP
	• DNS (53)

Config:

```
R1(config-ext-nacl)#deny tcp any host 1.1.1.1 eq ?
<0-65535>      Port number
bgp            Border Gateway Protocol (179)
chargen       Character generator (19)
cmd           Remote commands (rcmd, 514)
daytime       Daytime (13)
```

Destination Port Example:

```
R1(config-std-nacl)#deny tcp any host 1.1.1.1 eq 80
```

Explanation:

- Protocol = TCP
- Source = any
- Destination = 1.1.1.1
- Destination port = 80

Effect:

All HTTP traffic to 1.1.1.1 is denied.

Note:

After the destination IP address and/or destination port numbers, there are many more options you can use to match (not necessary for the CCNA).

Some examples:

- **ack**: match the TCP ACK flag
- **fin**: match the TCP FIN flag
- **syn**: match the TCP SYN flag
- **ttl**: match packets with a specific TTL value
- **dscp**: match packets with a specific DSCP value

Important Rule:

If you specify:

1. Protocol
2. Source IP
3. Source port
4. Destination IP
5. Destination port

Then the packet must match **all** these values to match that ACL entry.

If even one value does not match, the packet will not match that entry.

Advanced Practice

1. Allow traffic from 10.0.0.0/16 to access the server at 2.2.2.2/32 using HTTPS.

```
R1(config-ext-nacl)#permit tcp 10.0.0.0 0.0.255.255 2.2.2.2 0.0.0.0 eq 443
```

Explanation:

- Protocol = TCP (HTTPS uses TCP)
- Source = 10.0.0.0/16
- Destination = 2.2.2.2
- Destination port = 443

Only HTTPS traffic is allowed.

2. Prevent all hosts using source UDP port numbers from 20000 to 30000 from accessing the server at 3.3.3.3/32.

```
R1(config-ext-nacl)#deny udp any range 20000 30000 host 3.3.3.3
```

Explanation:

- Protocol = UDP
- Source = any
- Source port = 20000 to 30000
- Destination = 3.3.3.3

Only UDP traffic using those source ports is denied.

3. Allow hosts in 172.16.1.0/24 using a TCP source port greater than 9999 to access all TCP ports on server 4.4.4.4/32 except port 23.

```
R1(config-ext-nacl)#permit tcp 172.16.1.0 0.0.0.255 gt 9999 host 4.4.4.4 neq 23
```

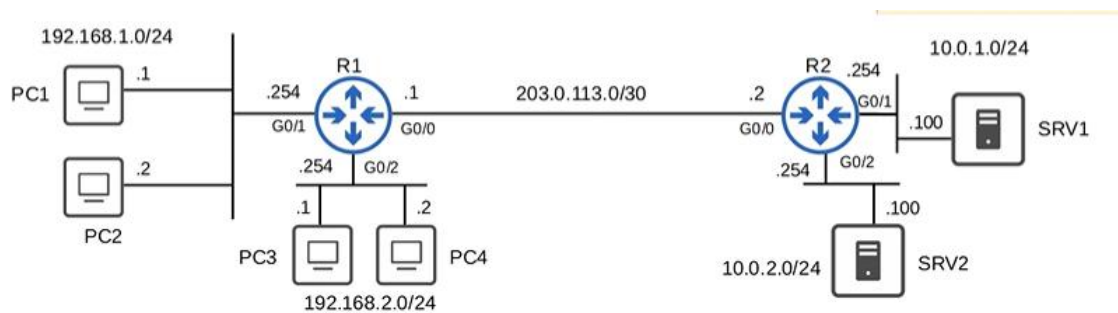
Explanation:

- Protocol = TCP
- Source network = 172.16.1.0/24
- Source port = greater than 9999
- Destination = 4.4.4.4
- Destination port = anything except 23

This is a very specific rule. That is the power of extended ACLs.

Applying Extended ACLs

Network Topology:



Requirements:

1. Hosts in 192.168.1.0/24 cannot use HTTPS to access SRV1 (10.0.1.100).
2. Hosts in 192.168.2.0/24 cannot access 10.0.2.0/24.
3. Hosts in 192.168.1.0/24 and 192.168.2.0/24 cannot ping 10.0.1.0/24 or 10.0.2.0/24.

All ACLs will be configured on R1.

Rule for Applying Extended ACLs:

Extended ACLs should be applied as close to the source as possible, so packets are dropped early and do not waste network resources.

(Standard ACLs are applied close to the destination because they are less specific.)

Requirement 1: Block HTTPS from 192.168.1.0/24 to SRV1

ACL Configuration:

```
R1(config)#ip access-list extended HTTP_SRV1
R1(config-ext-nacl)#deny tcp 192.168.1.0 0.0.0.255 host 10.0.1.100 eq 443
R1(config-ext-nacl)#permit ip any any
R1(config-ext-nacl)#interface g0/1
R1(config-if)#ip access-group HTTP_SRV1 in
```

Explanation:

- deny tcp → match TCP traffic.
- Source: 192.168.1.0/24
- Destination: host 10.0.1.100 (SRV1)
- eq 443 → HTTPS port
- Then permit ip any any allows all other traffic.

Why G0/1 inbound?

- Source network is 192.168.1.0/24.
- Applying inbound on R1 G0/1 drops the HTTPS packet immediately when it reaches R1.

Requirement 2: Block 192.168.2.0/24 from Accessing 10.0.2.0/24

ACL Configuration:

```
R1(config)#ip access-list extended BLOCK_10.0.2.0/24
R1(config-ext-nacl)#deny ip 192.168.2.0 0.0.0.255 10.0.2.0 0.0.0.255
R1(config-ext-nacl)#permit ip any any
R1(config-ext-nacl)#interface g0/2
R1(config-if)#ip access-group BLOCK_10.0.2.0/24 in
```

Explanation:

- deny ip → match all IP traffic.
- Source: 192.168.2.0/24
- Destination: 10.0.2.0/24
- Then permit all other traffic.

Why G0/2 inbound?

- Source network is 192.168.2.0/24.
- Applying inbound on G0/2 blocks the traffic immediately at the source side.

Now requirement 2 is fulfilled.

Requirement 3: Prevent Ping (ICMP)

Ping uses ICMP.

Hosts in:

- 192.168.1.0/24
- 192.168.2.0/24

Must not ping:

- 10.0.1.0/24
- 10.0.2.0/24

ACL Configuration:

```
R1(config)#ip access-list extended BLOCK_ICMP
R1(config-ext-nacl)#deny icmp 192.168.1.0 0.0.0.255 10.0.1.0 0.0.0.255
R1(config-ext-nacl)#deny icmp 192.168.1.0 0.0.0.255 10.0.2.0 0.0.0.255
R1(config-ext-nacl)#deny icmp 192.168.2.0 0.0.0.255 10.0.1.0 0.0.0.255
R1(config-ext-nacl)#permit ip any any
R1(config-ext-nacl)#interface g0/0
R1(config-if)#ip access-group BLOCK_ICMP out
```

Explanation:

Three deny entries:

1. Block 192.168.1.0 → 10.0.1.0 (ICMP)
2. Block 192.168.1.0 → 10.0.2.0 (ICMP)
3. Block 192.168.2.0 → 10.0.1.0 (ICMP)

There is no entry for 192.168.2.0 → 10.0.2.0 because that traffic was already blocked in the previous ACL.

permit ip any any allows other traffic.

Why G0/0 outbound?

- Both source networks pass through G0/0 toward R2.
- Applying outbound on G0/0 blocks ping traffic from both networks toward the server networks.

Verifying the ACLs

```
R1#show access-lists
Extended IP access list BLOCK_10.0.2.0/24
 10 deny ip 192.168.2.0 0.0.0.255 10.0.2.0 0.0.0.255
 20 permit ip any any
Extended IP access list BLOCK_ICMP
 10 deny icmp 192.168.1.0 0.0.0.255 10.0.1.0 0.0.0.255
 20 deny icmp 192.168.1.0 0.0.0.255 10.0.2.0 0.0.0.255
 30 deny icmp 192.168.2.0 0.0.0.255 10.0.1.0 0.0.0.255
 40 permit ip any any
Extended IP access list HTTP_SRV1
 10 deny tcp 192.168.1.0 0.0.0.255 host 10.0.1.100 eq 443
 20 permit ip any any
```

This confirms all three ACLs are configured correctly.

Checking Which ACL is Applied to an Interface

```
R1#show ip interface g0/0
GigabitEthernet0/0 is up, line protocol is up
Internet address is 203.0.113.1/30
Broadcast address is 255.255.255.255
Address determined by non-volatile memory
MTU is 1500 bytes
Helper address is not set
Directed broadcast forwarding is disabled
Outgoing access list is BLOCK_ICMP
Inbound access list is not set
Proxy ARP is enabled
Local Proxy ARP is disabled
Security level is default
Split horizon is enabled
ICMP redirects are always sent
ICMP unreachable are always sent
ICMP mask replies are never sent
```

This shows:

- BLOCK_ICMP is applied outbound on G0/0.
- No inbound ACL on this interface.

You can also check running-config but show ip interface directly shows inbound and outbound ACL status.

Understanding

- Extended ACLs are applied close to the source.
- Each requirement was solved with a separate ACL.
- Direction (in/out) depends on traffic flow.
- Verification is done using:
 - show access-lists
 - show ip interface <interface>

What we Covered

1] Numbered ACL Configuration

- Traditional method (global config mode)
- Named ACL method (recommended)
- Modern IOS allows numbered ACL using named method
- Difference between traditional and named configuration

2] Editing ACLs

- Default sequence numbers (10, 20, 30...)
- Deleting individual ACL entries
- Inserting entries between existing rules
- Problem with old global method (deletes whole ACL)
- ACL resequencing command

3] Extended ACL Basics

- Extended ACL works top to bottom
- Matches:
 - Protocol (TCP, UDP, ICMP, IP)
 - Source IP address
 - Destination IP address
 - Port numbers (Layer 4)
- Extended ACL number ranges:
 - 100–199
 - 2000–2699

4] Protocol Matching

- Match by protocol name (tcp, udp, icmp, ip)
- Match by protocol number (6, 17, 1, etc.)
- ip means all IP traffic

5] Source and Destination Matching

- Use any
- Use host for single IP
- Use network + wildcard mask
- Must use host or 0.0.0.0 for single host

6] Port Number Matching

- Used with TCP and UDP
- Options: eq, gt, lt, range
- Common ports: 80 (HTTP), 443 (HTTPS), 23 (Telnet)

7] Advanced Extended ACL Rules

- Can match:
 - Source port
 - Destination port
- All conditions must match for rule to work
- Very specific traffic filtering possible

8] Applying Extended ACLs

- Applied close to the source
- Direction matters (inbound/outbound)
- Standard ACL applied near destination
- Extended ACL applied near source

9] Verification Commands

- show access-lists
- show ip interface <interface>

Final Understanding

Extended ACLs provide detailed, powerful, and specific traffic control with better editing flexibility using named ACL mode.